

```

create table sales_data_v2 ( p_type string, total_sales int ) row
format delimited fields terminated by ',';

load data local inpath 'file:///home/growdataskills/
sales_data_raw.csv' into table sales_data_v2;

# Command to create a new table from different table
create table sales_data_v2_bkup as select * from sales_data_v2;

# CSV SerDe
create table csv_table
(
name string,
location string
)
row format serde 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
with serdeproperties (
'separatorChar' = ',',
'quoteChar' = '\"',
'escapeChar' = '\\'
)
stored as textfile
tblproperties ("skip.header.line.count" = "1");

load data local inpath 'file:///home/growdataskills/csv_file.csv'
into table csv_table;


# JSON Serde
create table json_table
( name string,
id int,
skills array
)
row format serde 'org.apache.hadoop.hive.serde2.JsonSerDe'
stored as textfile;

load data local inpath 'file:///home/growdataskills/json_file.json'
into table json_table;

# Parquet file format
create table sales_data_pq_final
(
product_type string,
total_sales int
)
stored as parquet;

from sales_data_v2 insert overwrite table sales_data_pq_final select
*;

# create csv table for sales data

```

```
create table automobile_sales_data ( ORDERNUMBER int,
QUANTITYORDERED int, PRICEEACH float, ORDERLINENUMBER int, SALES
float, STATUS string, QTR_ID int, MONTH_ID int, YEAR_ID int,
PRODUCTLINE string, MSRP int, PRODUCTCODE string, PHONE string, CITY
string, STATE string, POSTALCODE string, COUNTRY string, TERRITORY
string, CONTACTLASTNAME string, CONTACTFIRSTNAME string, DEALSIZE
string ) row format delimited fields terminated by ','
tblproperties("skip.header.line.count"="1") ;
```

```
load data local inpath 'file:///home/growdataskills/
sales_order_data.csv' into table sales_order_data_csv_v1;
```

```
# load sales_order_data.csv data into above mentioned tables
create table automobile_sales_data_orc ( ORDERNUMBER int,
QUANTITYORDERED int, PRICEEACH float, ORDERLINENUMBER int, SALES
float, STATUS string, QTR_ID int, MONTH_ID int, YEAR_ID int,
PRODUCTLINE string, MSRP int, PRODUCTCODE string, PHONE string, CITY
string, STATE string, POSTALCODE string, COUNTRY string, TERRITORY
string, CONTACTLASTNAME string, CONTACTFIRSTNAME string, DEALSIZE
string ) stored as orc;
```

```
from automobile_sales_data insert overwrite table
automobile_sales_data_orc select *;
```

```
# Hive group by query
select year_id, sum(sales) as total_sales from
automobile_sales_data_orc group by year_id;
```

```
# Set this property if doing static partition
set hive.mapred.mode=strict;
```

```
# create table command for partition tables – for Static
create table automobile_sales_data_static_part
(
ORDERNUMBER int,
QUANTITYORDERED int,
SALES float,
YEAR_ID int
)
partitioned by (COUNTRY string);
```

```
# load data in static partition
insert overwrite table automobile_sales_data_static_part
partition(country = 'USA') select
ordernumber,quantityordered,sales,year_id from
automobile_sales_data_orc where country = 'USA';
```

```
# set this property for dynamic partitioning
set hive.exec.dynamic.partition.mode=nonstrict;
```

```
create table automobile_sales_data_dynamic_part
( ORDERNUMBER int,
QUANTITYORDERED int,
```

```

SALES float,
YEAR_ID int
) partitioned by (COUNTRY string);

# load data in dynamic partition table
insert overwrite table automobile_sales_data_dynamic_part
partition(country) select
ordernumber,quantityordered,sales,year_id,country from
automobile_sales_data_orc ;

# multilevel partition
create table automobile_sales_dynamic_multilevel_part
( ORDERNUMBER int,
QUANTITYORDERED int,
SALES float
) partitioned by (COUNTRY string, YEAR_ID int);

# load data in multilevel partitions
insert overwrite table automobile_sales_dynamic_multilevel_part
partition(country,year_id) select
ordernumber,quantityordered,sales,country ,year_id from
automobile_sales_data_orc;

# Create users and locations table

create table users
(
    id int,
    name string,
    salary int,
    unit string
)
row format delimited
fields terminated by ',';

load data local inpath 'file:///home/growdataskills/users.csv' into
table users;

create table locations
(
    id int,
    location string
)
row format delimited
fields terminated by ',';

load data local inpath 'file:///home/growdataskills/locations.csv'
into table locations;

# Set Properties and Command to create buckets
set hive.enforce.bucketing=true;

create table buck_users

```

```

(
    id int,
    name string,
    salary int,
    unit string
)
clustered by (id)
into 2 buckets;

insert overwrite table buck_users select * from users;

# Drop bucket table
drop table buck_users

# Create tables with buckets for optimized joins
create table buck_users
(
    id int,
    name string,
    salary int,
    unit string
)
clustered by (id)
sorted by (id)
into 2 buckets;

insert overwrite table buck_users select * from users;

create table buck_locations
(
    id int,
    location string
)
clustered by (id)
sorted by (id)
into 2 buckets;

insert overwrite table buck_locations select * from locations;

# Map Side Join
SET hive.auto.convert.join=true;

SELECT * FROM buck_users u INNER JOIN buck_locations l ON u.id =
l.id;

# Bucket Map Join
set hive.optimize.bucketmapjoin=true;
SET hive.auto.convert.join=true;

SELECT * FROM buck_users u INNER JOIN buck_locations l ON u.id =
l.id;

```

```
# Sorted Merge Bucket Map Join
set hive.auto.convert.sortmerge.join=true;
set hive.optimize.bucketmapjoin.sortedmerge = true;

SELECT * FROM buck_users u INNER JOIN buck_locations l ON u.id =
l.id;
```